

时间轮

背景

在各种业务场景中，我们总是会需要一些定时进行一些操作，这些操作可能是需要在指定的某个时间点操作，也可能是每过一个固定的时间间隔后进行操作，这就要求我们需要有一个定时调度的逻辑，同时，这种定时操作，既有可能在某一时刻数量比较密集，也有可能时间间隔比较密集，这就要考虑定时调度器对性能的影响。

在本次考试中， 你需要使用时间轮实现一个对于**周期性**任务的调度程序。

时间轮

总的任务调度器由若干个轮状结构构成，在之后的描述中， 时间轮指代这种轮状结构。

一个调度器可以包含多级时间轮， 通常使用时间单位来划分不同层次的时间轮， 比如秒级时间轮， 分钟级时间轮， 小时级时间轮等等（你可以想象把一个钟表拆成 3 个表盘）。

在本次机考题中， 你需要完成的调度器包含秒分时三个时间轮， 同现实生活。

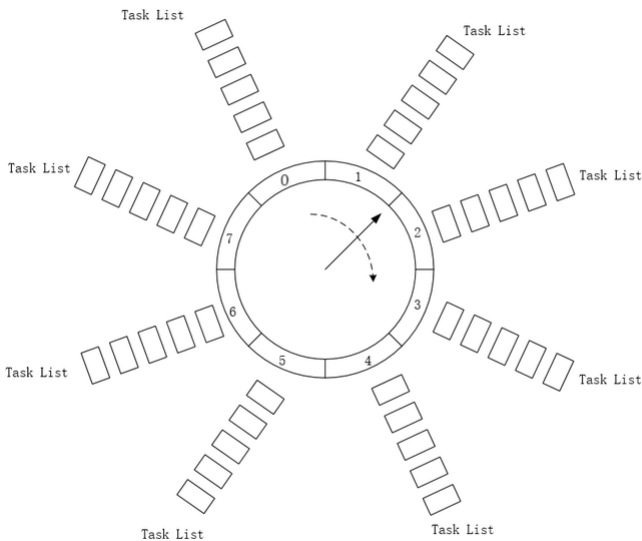
时间轮由排成环形的若干个时间格构成， 每个时间格代表了当前时间轮的一个基本时间间隔(Interval)。

时间格的数量表示了该时间轮处理的时间间隔的个数。

每个时间格又称作一个槽位(slot)， 每个槽记录了一个任务列表(Task List)， 表示在对应时间间隔内需要触发的任务。

任务列表中的每个节点(TaskNode)对应着每个任务。

下面是一个时间轮的示意图（槽位数为 8 仅作为示意， 你需要实现的相关参数见[时间轮的参数](#)）：



时间轮的参数

在本次机考中， 你需要实现的时间轮同现实生活， 取 1s 为最小单位时间， 则

秒级时间轮槽位(slot)数为 60， 时间间隔(Interval)为 1s；

分钟级时间轮槽位(slot)数为 60， 时间间隔(Interval)为 1min = 60s；

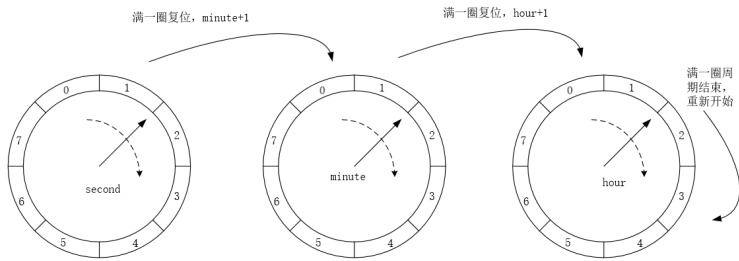
小时级时间轮槽位(slot)数为 24， 时间间隔(Interval)为 1h = 60min = 3600s。

每个槽位内都有一个任务列表， 存放在该时间间隔内需要触发的任务。

时间轮的状态切换

类似钟表的表针，
时间轮会维护一个值表示当前所在的槽位， 每过一个基本时间间隔， 就将这个值修改为下一个槽位。

对于秒级时间轮， 每过一秒切换至下一槽位。
而在多级时间轮中， 次一级的时间轮遍历一遍所有槽位， 即走完一圈后， 本级时间轮切换至下一槽位。
如 60 秒后， 秒级时间轮切换 60 次回到原位， 而分钟级时间轮切换至下一槽位， 类似于加法的进位。



在切换到某一槽位后， 时间轮会对对应位置任务列表内的所有任务进行处理。
如果当前为最低级时间轮， 比如秒级， 则表示对应任务列表内的所有任务都需要在此时刻触发。
而如果不是在最低级时间轮， 则表示这些任务将在对应的这段时间内触发， 应移动到低一级时间轮上更精确的槽位。
如若为分钟级， 则表示其需要在该分钟内触发， 需要把所有任务移动到秒级时间轮内对应槽位的任务列表。

对于一个任务， 它会在某一时刻被加入时间轮， 并在一段时间 (first_interval) 后触发。在第一次触发后， 它将周期 (period) 性地反复触发。请注意， 虽然文中将会描述为任务的触发， 但在具体实现时， 你只需要在每个时刻返回应该被触发的任务（见代码框架）， 而无需关心任务的具体触发。

在[题目最后](#)， 我们将用一些示例说明时间轮是如何进行任务调度的， 并给出相关计算的伪代码。

考试任务

本次机考中， 你需要完成对于时间轮以及相关类的实现。

具体要求如下：

- 支持动态添加任务、删除任务
- 支持任务的周期性触发
- 使用链表维护任务列表

在下发的 `Timer.hpp` 中提供了对于程序的框架， 你只需要实现其中的相关类和函数。

代码介绍

在本次机考中， 涉及到的类有 `Task`， `TaskNode`， `TimingWheel`， `Timer`。

总体来说， 我们通过 `Timer` 类来管理所有时间轮 `TimingWheel` 和任务 `Task`， 对于时间轮， 每个槽位都有一个链表， 链表上的节点为 `TaskNode`。

一种可能的大体架构如下：

- `TaskNode* addTask(Task* task)`：将一个任务加入时间轮中，返回该任务在时间轮上任务列表内的节点。
- `void cancelTask(TaskNode *p)`：取消一个任务，将其从时间轮上移除。保证删除的任务存在。
- `std::vector<Task*> tick()`：最低时间轮（秒级）前进一格，返回在此时刻需要执行的任务列表，无需关心列表中的任务顺序。
需要注意时间轮的进位。

同样地，你可以在规定的范围内自由增加需要的成员变量和成员函数。

注意

- 在本次机考题中，以整数 1 作为最小时间单位。
- 在本次机考中，你需要实现的时间轮即现实生活的时分秒三个时间轮，相关参数见[时间轮的参数](#)，不允许修改时间轮的参数。
- 除了 `Timer::tick()` 返回的 `std::vector<Task*>` 外，不得使用任何 STL 容器。
- 不得使用除题目给定外的其他头文件。
- 可以在题目允许的范围内，自行修改框架给出的架构。
- 时间轮最初处于 `slot = 0`，且在第一次 `tick()` 后，时间轮会处于 `slot = 1`，并返回此时（1 时刻）需要触发的任务（可参考样例）。
- 数据保证 `getFirstInterval()` 和 `getPeriod()` 返回值均为正整数。
- 如果任务下一次触发的时间超出时间轮范围，则不需要再进行处理，但需要将其对应的节点析构以避免内存泄漏。

样例

`sample.cpp` 将会使用你完成的 `Timer.hpp` 进行任务调度，具体内容同[示例](#)。你可以通过其输出判断你的程序的正确性，也可以对其进行修改来调试。

测试点

你只需要在 OJ 上提交实现的 `Timer.hpp`。

测试点分三个阶段，你可以参考以获得部分分：

- 你只需要考虑任务集中在第一个（秒级）时间轮上。
- 你需要考虑三个时间轮。
- 你需要保证没有内存泄漏。

数据点	数据特征
1	所有任务 <code>period=60</code>
2 ~ 3	任务集中在第一个时间轮上
4 ~ 5	没有 <code>cancelTask</code>
6 ~ 7	综合测试
8 ~ 10	压力测试，数据量较大

数据点	数据特征
11 ~ 20	同上，检查内存泄露

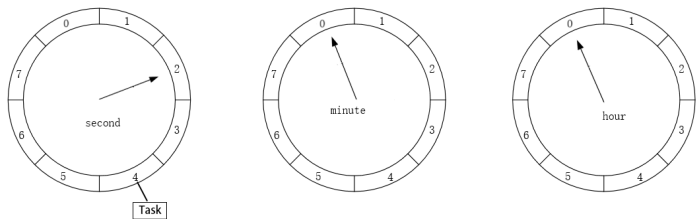
其中测试点 1 ~ 10 分数占比 80%，测试点 11 ~ 20 占比 20%。

附件说明

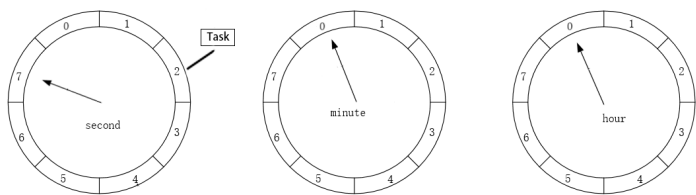
- `Timer.hpp`：你需要实现的文件
- `Task.hpp`：任务类的样例实现，你不需要过于关心
- `sample.cpp`：样例程序

示例

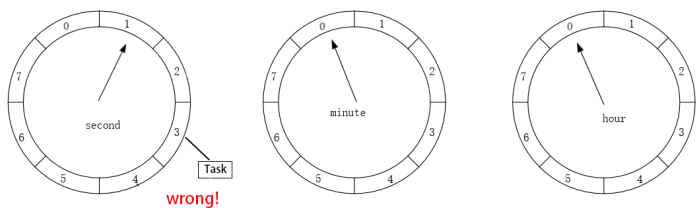
假设我们有三个时间轮，槽位数都是 8（如下图）。那么在这个例子中，1min 等于 8s，1h = 8min。最初，所有时间轮都位于 0。在开始的 1s，我们什么都不做。当秒级时间轮走到 2 时，我们加入一个 `first_interval = 2, period = 3` 的任务 Task，则它首先会被加到秒级时间轮中 4 的位置。



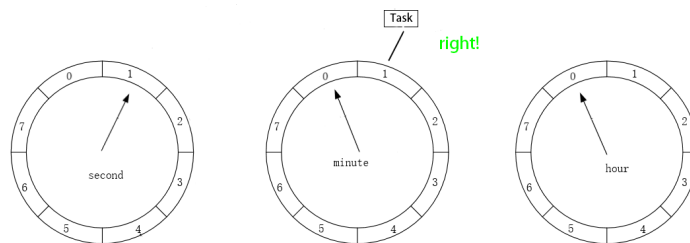
当秒级时间轮再走过两次，到达 4 位置后，Task 应该被触发。由于 `period = 3`，它应该在 3s 后再次被触发。因此，它会被移到秒级时间轮上 7 的位置等待被触发。同理，这次被触发后，应将它再移到秒级时间轮 2 的位置（时间轮是循环走的）。



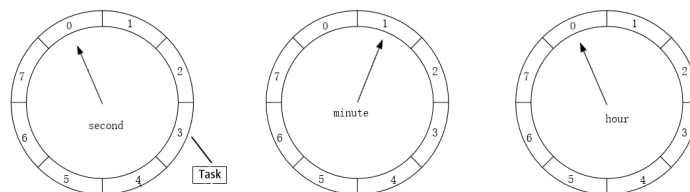
而当一个任务离执行的时间间隔大于当前时间轮的最高范围时，便要将它放到更高一级的时间轮上。比如在 1s 时，我们加入一个 `first_interval = 10, period = 9` 的任务 Task。此时若想把它放在秒级时间轮上，则应该放在秒级时间轮 3 的位置，但这将与 2s 后需要触发的任务混淆。



所以正确的做法应该是，将其放到分钟级时间轮 1 的位置。



当 7s 后秒级时间轮走完一圈回到 0 时，将触发分钟时间轮的进位，分钟时间轮走到 1。此时，Task 将被下放到秒级时间轮上 3 的位置。



当再过 3s，秒级时间轮走到 3 时，Task 将被触发。由于 $\text{period} = 9$ ，它需要在 9s 后再次被触发，也超出了秒级时间轮的范围。于是它将再次被放到分钟级时间轮 2 的位置，再以此类推。同样地，如果超过分钟级时间轮的范围，那就需要被放在小时级时间轮上。而**如果任务超出了所有时间轮能控制的范围，则不需要处理该任务，但在具体实现时，你需要将其对应的节点析构以避免内存泄漏**。但如果 first_interval 未超过而 period 超过，则该任务将会被触发一次。

特别地，对于间隔刚好等于当前时间轮范围的情况，任务应该被放在当前时间轮上。例如，在 1s 时，加入一个 $\text{first_interval} = 8, \text{period} = 8$ 的任务 Task，则 Task 应该被放在秒级时间轮上 1 的位置。则在秒级时间轮再转一圈到 1 时，Task 将会被触发。之后由于 $\text{period} = 8$ ，则它将再次被放在秒级时间轮 1 的位置。

通过上面的描述，我们可以得出任务放置对应槽位的计算方法：假设任务 `task` 距离下一次触发的时间为 `time` ($= \text{first_interval}$ 或 period)，当前时间轮的槽位为数组 `slots`，总数为 `size`，每个槽位的时间间隔 `interval`，当前正指向 `current_slot`，则有以下伪代码：

```
if time / interval <= size then
    insert task to slots[(current_slot + time / interval) % size]
else
    time = time + current_slot * interval //由于下放的时候当前时间轮会指向 0，所以往上放
    的时候加上偏移量，使得 task 相当于在当前时间轮指向 0 的时候放置
    move task to the bigger timingwheel
```

同样地，当前时间轮把当前槽位中的任务下放到更低一级时间轮上，也有以下伪代码：

```
for task in tasklist
    task.time %= interval //距离 task 被放上当前时间轮已经过了 interval 的整数倍，上放时
    加上的偏移量保证了这一点
    move task to the smaller timingwheel
```